

README

digital.vasic.containers

A generic, reusable Go module for container orchestration, health checking, lifecycle management, and service discovery. Supports Docker, Podman, and Kubernetes runtimes.

Installation

```
go get digital.vasic.containers
```

Quick Start

```
package main

import (
    "context"
    "fmt"
    "log"

    "digital.vasic.containers/pkg/boot"
    "digital.vasic.containers/pkg/endpoint"
    "digital.vasic.containers/pkg/health"
    "digital.vasic.containers/pkg/logging"
    "digital.vasic.containers/pkg/runtime"
)

func main() {
    ctx := context.Background()

    // Auto-detect container runtime (Docker or Podman)
    rt, err := runtime.AutoDetect(ctx)
    if err != nil {
        log.Fatal(err)
    }
    fmt.Printf("Using runtime: %s\n", rt.Name())

    // Define service endpoints
```

```

endpoints := map[string]endpoint.ServiceEndpoint{
    "postgres": endpoint.NewEndpoint().
        WithHost("localhost").WithPort("5432").
        WithHealthType("tcp").WithRequired(true).
        WithComposeFile("docker-compose.yml").
        WithServiceName("postgres").
        Build(),
    "redis": endpoint.NewEndpoint().
        WithHost("localhost").WithPort("6379").
        WithHealthType("tcp").WithRequired(true).
        WithComposeFile("docker-compose.yml").
        WithServiceName("redis").
        Build(),
}

// Boot all services
mgr := boot.NewBootManager(endpoints,
    boot.WithRuntime(rt),
    boot.WithLogger(logging.NewSlogAdapter()),
)

summary, err := mgr.BootAll(ctx)
if err != nil {
    log.Fatalf("Boot failed: %v", err)
}

fmt.Printf("Started: %d, Failed: %d\n",
    summary.Started, summary.Failed)
}

```

Features

- **Multi-runtime support:** Docker, Podman, Kubernetes
- **Auto-detection:** Automatically finds available container runtime
- **Health checking:** TCP, HTTP, gRPC, and custom health checks with retry
- **Compose orchestration:** Batch operations grouped by compose file/profile
- **Lifecycle management:** Lazy boot, idle shutdown, concurrency semaphores
- **Resource monitoring:** System and per-container CPU/memory/disk, cluster snapshots
- **Event system:** Publish/subscribe for 20 lifecycle event types
- **Service discovery:** TCP port probe and DNS-based discovery
- **Prometheus metrics:** Built-in metrics collection
- **Pluggable logging:** Bring your own logger (slog adapter included)
- **Remote distribution:** Distribute containers across multiple hosts via SSH

- **Resource-aware scheduling:** 5 strategies (resource_aware, round_robin, affinity, spread, bin_pack)
- **SSH tunnel management:** Cross-host networking with auto port allocation
- **Remote volumes:** SSHFS, NFS, and rsync-based volume sharing
- **Automatic failover:** Detect offline hosts and reschedule containers
- **Environment configuration:** .env files and CONTAINERS_REMOTE_* env vars

Architecture

boot.BootManager	
└─ compose.ComposeOrchestrator	(Docker Compose operations)
└─ health.HealthChecker	(TCP/HTTP/gRPC checks)
└─ discovery.Discoverer	(Service discovery)
└─ distribution.Distributor	(Remote distribution)
└─ event.EventBus	(20 lifecycle event types)
└─ metrics.MetricsCollector	(Prometheus metrics)
└─ logging.Logger	(Pluggable logging)
distribution.Distributor	
└─ scheduler.Scheduler	(5 placement strategies)
└─ remote.HostManager	(Host registry + probing)
└─ remote.RemoteExecutor	(SSH command execution)
└─ network.TunnelManager	(SSH tunnels)
└─ volume.VolumeManager	(SSHFS/NFS/rsync)
lifecycle.LifecycleManager	
└─ LazyBooter	(Start on first Acquire)
└─ IdleShutdown	(Stop after inactivity)
└─ ConcurrencySemaphore	(Limit parallel users)
runtime.ContainerRuntime	
└─ DockerRuntime	
└─ PodmanRuntime	
└─ KubernetesRuntime	
└─ remote.RemoteRuntime	(ContainerRuntime over SSH)

Remote Distribution

Distribute containers across local and remote hosts. See [docs/REMOTE_DISTRIBUTION.md](#) for the full guide.

```
import (
    "digital.vasic.containers/pkg/distribution"
    "digital.vasic.containers/pkg/envconfig"
    "digital.vasic.containers/pkg/remote"
```

```

    "digital.vasic.containers/pkg/scheduler"
)

// Load remote host configuration from .env
cfg, _ := envconfig.LoadFromEnv()
hosts := cfg.ToRemoteHosts()

// Create host manager and register hosts
hm := remote.NewDefaultHostManager(remote.DefaultOptions())
for _, h := range hosts {
    hm.AddHost(h)
}

// Create distributor
dist := distribution.NewDistributor(
    distribution.WithScheduler(
        scheduler.NewDefaultScheduler(hm),
    ),
    distribution.WithHostManager(hm),
    distribution.WithExecutor(
        remote.NewSSHExecutor(remote.DefaultOptions()),
    ),
)

// Distribute containers
summary, _ := dist.Distribute(ctx,
    []scheduler.ContainerRequirements{
        {Name: "web", Image: "nginx:latest"},
        {Name: "cache", Image: "redis:latest"},
    },
)

fmt.Printf("Local: %d, Remote: %d\n",
    summary.LocalContainers, summary.RemoteContainers)

```

Service Orchestrator

Auto-discover and manage all containerized services with automatic remote distribution:

```

import (
    "digital.vasic.containers/pkg/orchestrator"
    "digital.vasic.containers/pkg/compose"
    "digital.vasic.containers/pkg/remote"
)

// Create orchestrator with local compose and optional remote
// support
orch := orchestrator.New(
    orchestrator.WithLocalOrchestrator(composeOrch),
    orchestrator.WithRemoteExecutor(remoteExec), // optional
    orchestrator.WithHostManager(hostMgr),       // optional
)

```

```

    orchestrator.WithProjectDir("/path/to/project"),
)

// Auto-discover all docker-compose files in docker/ directory
orch.DiscoverServices("docker")

// Or manually add services
orch.AddService(orchestrator.Service{
    Name:      "mcp",
    ComposeFile: "docker/mcp/docker-compose.mcp-servers.yml",
    Description: "MCP servers (32+ servers)",
})

// Start all services (remote if configured, local otherwise)
err := orch.StartAll(ctx)

// Start a specific service
err := orch.StartService(ctx, "mcp")

// List discovered services
services := orch.ListServices()

```

When remote distribution is enabled (both RemoteExecutor and HostManager provided), all services are automatically deployed to the remote host with automatic fallback to local.

Container Monitoring (ctop)

Real-time container monitoring with top/htop-style display for local and remote containers:

```

import (
    "context"
    "digital.vasic.containers/pkg/ctop"
    "digital.vasic.containers/pkg/remote"
)

// Create collector with optional remote host support
collector := ctop.NewCollector("podman", hostManager)

// Collect container data
list, _ := collector.Collect(context.Background())
fmt.Printf("Containers: %d running, %d stopped\n", list.Running,
    list.Stopped)

// Create interactive display
display := ctop.NewDisplay(collector, ctop.DefaultDisplayConfig())

// Run interactive TUI (blocks until quit)
display.Run(context.Background())

```

```
// Or get a snapshot
snapshot, _ := display.RenderSnapshot(context.Background())
fmt.Println(snapshot)

// Or get JSON output
json, _ := display.RenderJSON(context.Background())
fmt.Println(json)
```

CLI Usage

```
# Install the ctop CLI
go install digital.vasic.containers/cmd/ctop@latest

# Run interactive monitoring
ctop

# One-time snapshot
ctop --once

# JSON output
ctop --json

# Filter by host
ctop --host thinker

# Sort by memory
ctop --sort mem

# Show stopped containers
ctop --all
```

Display Features

- **Color-coded resource usage:** Green (low) → Yellow (medium) → Red (high)
- **Sorting:** CPU, memory, name, state, uptime, runtime, host
- **Filtering:** By host name, container name, running/stopped state
- **Multi-host:** Shows containers from local and remote hosts
- **Remote support:** Integrates with HostManager for distributed monitoring

License

MIT